With relabeling, you can add, remove, or modify labels on a per-target basis. A concrete example would be to rename a label to standardize on a label name. Let's assume our source would expose the URL path under the label `handler`, but we want to use `path` in our setup. In this case, we want to do the following relabeling:

```
api_requests_total{method="GET", handler="/some/interesting/thing"}

=>

api_requests_total{method="GET", path="/some/interesting/thing"}
```

You can do this with a relabeling rule that you would define in your Prometheus scrape config (https://prometheus.io/blog/2015/06/01/advanced-service-discovery/#scrape-configurations-and-relabeling), like the following:

```
- source_labels: [handler]
  action: replace
  target_label: path
- regex: "handler"
  action: labeldrop
```

Matches the handler label name and adds the path label with the same label value

Once done, it drops the handler label name, leaving path as the one label capturing the URL path.

Tip

If you want to try out relabeling interactively, you can visit https://relabeler.promlabs.com/ and use the relabel rule (in the left-hand Relabeling Rules input box) and the following as an input for the right-hand side (Object Labels):

```
method: "GET"
handler: "/some/interesting/thing"
```

When you now click the Analyze Rules button, you should see the rule engine in action, yielding the expected result.

Further, to improve query performance, you can define recording rules (http://mng.bz/V1vX) in your Prometheus scrape config. This feature allows us to precompute frequently used or expensive expressions, effectively trading space for time, saving the result as a new time series. Let's assume we want to use a recording rule for capturing the server-side error rate of our API invocation example (with `api_requests_total` being a counter metrics type):

The recording rule name

```
- name: api_invocation_server_error
  interval: 5m
  rules:
  - record: http_status_5xx:api_requests_total:ratio_rate_5m
    expr:
      sum(rate(api_requests_total{code=~"5.."}[5m]))
        /
      sum(rate(api_requests_total[5m]))
```

How often to evaluate the rule (here, every 5 minutes)

The name of the resulting metric (note the usage of :, which is, by convention, reserved for recording rule metrics)

The PromQL expression to evaluate at the current time, resulting in a new time series with the metric name defined by the preceding record value—http_status_5xx:api_requests_total:ratio_rate_5m, in our case